Architecture · LangGraph · Amazon Bedrock AgentCore

Building an agent product as a Forward Deployed Engineer

Nine layers, in the order an FDE builds them for a client: from a backend that owns every business rule to a copilot that acts for each signed-in user. For each layer: what the client asked for, what the FDE asked the client to decide, how it works, how it was proved, and when to skip it.

1. The FDE's way of working
2. A backend that owns every rule
3. The harness: a queue, and recovery
4. Durable state
5. A swarm, and evaluations
6. Sign-in, and an agent that improves
7. Security before the cloud
8. Context, memory and a gateway
9. The agent on a managed runtime
10. A copilot for the users
11. What carries to the next client

VerBodh

BEFORE THE LAYERS

The FDE's way of working

A Forward Deployed Engineer works inside the client's business. The job is to turn what the business needs into an agent system that runs in production, in the client's own cloud, under the client's own rules.

THE EXAMPLE CLIENT

The example in this article is a travel insurer. Three claims handlers work a queue by hand: each reads the customer's story and documents, checks them against the policy and the policy wording, and approves with a payout, declines or refers. The head of claims operations wants an agent that works the queue the way a handler does: read the claim, the policy, the customer's history and the right sections; decide; work out the payout; and explain the decision in two or three sentences that cite the section. The goal, in the client's words: handle claims straight through, as accurately as the handlers do.

ONE BRIEF, ONE PLAN, ONE REVIEW

Every layer starts as a brief from someone at the client: claims operations, IT, information security, data governance or the platform team. The FDE does not start coding from the brief. The loop is the same each time:

```
brief from the client
  → plan: phases, each with checks the FDE can verify alone
  → "Choices for you": the decisions the documents don't make
  → STOP: the client reviews the plan
  → build phase by phase: tests → checks → tick → commit
  → live runs on the real model, with what they cost
  → documents brought up to date
```

- **Every check can be verified without a person:** a test, a command's output, an API response, or a page checked in a browser while signed in. Each is a checkbox in the plan.
- **The choices are the client's.** When a brief is silent, the plan lists the options, a recommendation and the reason, then waits. Examples later in this article: which database the cloud can reach, how the agent finds a document section, what to spend per improvement round.
- **Tests never call a model.** A stub chat model plays a scripted list of replies, and the agent's tools call the real backend in process. Stand-ins replace the identity provider, the judge

model, the gateway and the agent service. The whole suite runs with no keys and no network.

- **Thresholds are never lowered to get a green run.** Coverage stays at 80%. If a rule looks wrong, the FDE stops and asks.
- **Live runs have a budget.** Each live check reports what it spent. Across the layers below, a full live pass cost cents, not dollars.
- **The answer key stays apart.** The handlers' past decisions are the evaluation answer key. The agent can never read them, and nothing reads the held-out set's decisions except the scorer.

LAYER 1

A backend that owns every rule

The pilot: a backend, an agent and a dashboard. Every later layer builds on it.

THE CLIENT'S ASK

- A backend that holds the claims queue and enforces the payment rule: any payout over the approval limit ($1,000 at first) needs a handler's sign-off.
- An agent that decides one claim at a time, and a dashboard where handlers watch the queue, read each decision, watch the agent work and sign off large payouts.
- Out of scope for the pilot: real payments, messages to customers, sign-in.

HOW IT WORKS

- **The rule lives in the payment route, not in the agent's instructions.** A mistaken or misled agent still can't pay. The agent can only ask; the backend decides.
- **Calls go one way.** The dashboard and the agent both call the backend over HTTP. The backend never imports the agent, and the dashboard only shows what the agent did.
- **Every route has an operation id, a one-line description and no optional inputs**, because those become the agent's tool names and tool descriptions. Paying is its own operation that takes the amount.
- **The agent gets six tools, the ones a handler would use**: read the claim, the policy, the history and the wording; record a decision; pay. There is no sign-off tool: only a person signs off.
- **Data minimisation.** The agent sees the customer's earlier claims, but not their address, phone or date of birth, because it doesn't need them to decide.
- **Instructions live in the backend** and are served by it, so the agent and the dashboard show the same text.
- **The runner posts each step** (tool call, tool result, final reply) to the claim. The dashboard polls every 2 seconds: simpler than websockets and fast enough to watch.
- Claim states: waiting → in progress → waiting for sign-off (over the limit) or decided. Payment is recorded apart from the state.

```
agent —HTTP→ backend ←HTTP— dashboard
  get claim, policy, history, wording
  → record decision
      ≤ $1,000 → decided → pay → paid
      > $1,000 → waiting for sign-off
                 pay → 403, back to the model as text
  handler signs off in the dashboard → paid
```

HOW IT WAS PROVED

- Approve $420 → decided. Approve $1,250 → waiting for sign-off; paying it before sign-off → 403.
- **The split-payment attack:** $1,000 then $250 on the same claim → both refused.
- A wrong amount → 422. Paying twice, or paying a declined claim → 409. A payout on a decline → 422.
- A stub-model run on one claim ends decided and paid $420 with at least 6 steps recorded; a refused payment comes back to the model as text instead of crashing the run.
- A test fails if any agent file reads the business data directly instead of through the backend.

TRADE-OFFS AND WHEN TO SKIP

SQLite through the standard library was enough for 25 claims and added no dependency; polling replaced websockets; the agent decided from the wording alone. The gap it left: a claim that fails partway stays stuck in progress. **Never skip this layer:** every later layer assumes the backend, not the model, holds the rules.

LAYER 2

The harness: a queue, and recovery

Builds on layer 1. Asked for by claims operations, with IT.

THE CLIENT'S ASK

- Claims will get harder: more documents, more rules, and decisions that follow the claims manual as well as the wording.
- The pilot's code does much of the organising and hands over the whole wording every time. The client wants the agent to organise its own work.
- One run works the whole queue in an order the agent chooses, with a cap for cheap trials and a summary at the end; a to-do list the team can watch; a fresh helper for each claim that reports back in one line; documents read by section; self-healing that never decides or pays twice.

DECISIONS PUT TO THE CLIENT

- **The team's own to-do tool, not the library's to-do middleware.** The middleware has three states and keeps the list inside the agent. The client needed a fourth state, "left for a handler", and a list the dashboard can show, so the backend stores it.
- **The backend owns the run rules.** The cap, the second-failure rule and the missed-claim check are backend refusals, like the payment limit, so a confused agent can't break them.
- **A fresh agent per claim, as a tool.** The queue agent's "work this claim" tool builds a new claim agent on a new thread and returns one line. It stops each step getting more expensive as the run goes on.
- **One claim at a time.** Parallel would be faster but would add locking to SQLite; it waits for layer 4.
- **Sections, not search.** Both documents already have numbered headings, so the agent lists the sections and reads the ones it needs. No search index is needed.

HOW IT WORKS

```
start run (cap)  → stuck claims go back to waiting, attempt + 1
queue agent: list waiting claims → set the whole to-do list
  for each claim: "work this claim" tool
    → new claim agent, new thread:
      start claim (refused past the cap) → read claim, policy,
      history → list sections → read section × n
      → record decision → pay
    → one line back, or "failed: …" and the claim is released
      (1st failure → waiting, 2nd → left for a handler)
  → finish run (refused while a waiting claim is under the cap,
    and the refusal names it, so the agent goes back)
```

- **A refusal is feedback that steers the agent.** When the agent tries to finish early, the backend refuses and names the claims it missed.
- **Errors stop at the tool boundary.** Any exception inside a claim becomes a string for the queue agent, so one bad claim never stops the run.
- **Model retries:** the library's retry middleware, 2 retries with backoff, and a 60-second model timeout.
- **An audit trail with no extra store:** the sections the agent read are derived from the claim's steps and shown on the claim page.

HOW IT WAS PROVED

- A 6th claim on a cap-5 run is refused; a release on a decided claim is refused; a stuck claim is reclaimed at the next start.
- With stub models for both agents: fail once then decided; fail twice then left for a handler; an early finish refused, the missed claim worked, then the finish accepted.
- Live, on a fresh database with a cap of 5: the to-do list filled and was ticked off, the steps showed the sections read, 5 claims ended and 20 stayed waiting.

TRADE-OFFS AND WHEN TO SKIP

Serial over parallel, and section browsing over search. **Skip it** when the agent does one thing per request and never works a queue.

LAYER 3

Durable state

Approvals that wait for a person, and work that survives a restart. Builds on layers 1–2. Asked for before the move to the cloud.

THE CLIENT'S ASK

- When the agent approves a payout over the limit, its work ends. A handler signs off later and the system pays, but the agent never comes back to the claim.
- A crash, a restart or Ctrl-C loses the claim's work, and the claim starts again from scratch.
- Also: payout rules in the backend with a tool for the agent (it gets payouts right far more often with one), a trace per claim, records in a hosted Postgres, and nothing done twice.

DECISIONS PUT TO THE CLIENT

- **The claim agent becomes a LangGraph `StateGraph` written by hand**, keeping what the middleware did: every step recorded, refusals returned to the model, at most 30 model calls.
- **Paying becomes a graph step, not a model tool.** The graph never compares the payout with the limit; it follows the claim's status, so the limit stays in one place, the backend.
- **Thread id = claim + try number.** A stopped claim keeps its thread and carries on; a claim released after an error gets a new thread and starts afresh. The same id names its trace session.
- **The sign-off is saved first, and the agent is told after.** The backend doesn't wait for the agent, and the sign-off stands if the agent can't be reached.
- **A connection budget:** at most 4 database connections per process, because the hosted plan allows 15 through its pooler across every process. With one run, the most in use is 8.

HOW IT WORKS

```
START → agent ⇄ tools (payout calculator, record decision)
  agent → route by the claim's status (read from the backend)
    not approved                → END
    approved, under the limit   → pay → END
    approved, needs sign-off    → wait node: interrupt()
       checkpoint saved, graph stops
       handler signs off → backend saves who and when
       → agent service resumes: Command(resume={handler})
       → pay → END
```

- **The wait is a node of its own.** When it resumes, LangGraph runs the interrupted node again from the top, so nothing with a side effect may come before `interrupt()` in that node.
- `PostgresSaver` **saves a checkpoint after every step**, with `durability="sync"`. The agent stays synchronous, because async Postgres doesn't work with the default Windows event loop.
- **Every backend write is safe to repeat.** A step can run again after a restart, so the same decision again returns the claim (a different one is 409), and the same payment again is treated as done.
- **A small agent service** takes "resume this claim", answers at once and works in a background thread. The backend calls it with a 5-second timeout and ignores a failure. The next run lists unfinished claims and carries them on.
- **One query layer for two databases:** SQLAlchemy Core, so each query is written once for SQLite in tests and Postgres in production.

HOW IT WAS PROVED

- Tests use an in-memory saver; a "restart" is a new graph built on the same saver. Stub-model tests cover the pause, sign-off → resume → paid once, a stopped claim carrying on with one decision, an error starting afresh.
- Live: a held claim signed off in the browser was paid within 5 seconds. A run killed after a claim's first steps resumed on the same thread with one decision. Most connections seen: 8.
- A defect found live: sign-off to payment took 13 seconds, because `localhost` tried IPv6 first and waited about a second per call. Switching to 127.0.0.1 brought it to 2 seconds.

TRADE-OFFS AND WHEN TO SKIP

Rejecting a held payout was left for a later layer. **Skip it** when nothing waits for a person and every run ends in seconds.

LAYER 4

A swarm, and evaluations

Builds on layers 1–3. Asked for by claims operations, with IT.

THE CLIENT'S ASK

- Claims arrive in batches; one at a time, a queue of 25 takes over 10 minutes. Choosing the next claim is a fixed rule that needs no model.
- Nobody can say how often the agent agrees with the handlers, what a claim costs or how long it takes, without reading traces.
- A dispatcher, problems put right, starting from the dashboard, cost and time per claim, evaluations on labelled sets, and a scoreboard.

HOW IT WORKS

```
dashboard: start dispatch (N agents) → backend → agent service
  409 while a run is going · 503 if the service is down
dispatcher graph (no model, no checkpointer):
  list claims (in progress first, then waiting, oldest first)
  → Send(claim) × claims → worker, max_concurrency = N
  rate limit or timeout → wait, carry on, up to 3 tries
  other error → released, retried once on a new thread
  all back → finish run → summary built from the records
```

- **The queue agent is gone.** Picking the next claim is a rule, so a model-free graph fans out one LangGraph `Send` per claim.
- **The worker calls one harness function**, the only place the dispatcher touches the claim agent, so the agent can later move to the cloud.
- **Each claim keeps its own thread**, never a shared thread or a subgraph, because a claim can wait days for a sign-off.
- **One `PostgresSaver` per claim run** on a shared pool: a saver holds a lock around every read and write, so one saver for 25 agents would make them take turns.
- **Evaluations work copies** through the same dispatcher and agent, so live data is never touched. Copies never show in the queue and can't be signed off.
- **The answer key sits in tables no claim operation reads**, and the backend scores from its own records.

- **Cost comes from the provider's usage cost on each reply**; time is measured around each model call, so a sign-off wait is never counted.

SCOREBOARD MEASURE	DEFINITION
Accuracy	the decision matches the handler's and the payout is within $1
Cost of mistakes	wrongful decline $1,500; wrongful approval its payout; needless referral $50; wrong amount the difference
Straight-through rate	the share of claims not referred
Cost and time	model cost per claim, time per claim, run time

HOW IT WAS PROVED

LIVE CHECK	RESULT
Dispatch at N = 25	35 s for the run, against 364 s of summed claim time
A dispatch killed mid-run	the next one carried its 3 claims on as the same attempts, one decision each
Evaluation set, N = 25	80% right · 92% straight through · $2,175 cost of mistakes · $0.0012 a claim · 43 s
Peak database connections	26 before a pool fix, 10 after

- A library behaviour found in the build: a node's error handler did not catch a failure when several `Send` tasks ran in parallel. The worker now catches the two retryable errors itself. Same result, different mechanism.

TRADE-OFFS AND WHEN TO SKIP

At N = 25 each claim took 24.3 s against 16.5 s at N = 3: requests wait for the backend's 4 connections. The fix is fewer, larger backend calls per claim. **Skip it** when items go one at a time and nobody needs a quality score.

LAYER 5

Sign-in, and an agent that improves

Builds on layers 1–4. The dashboard goes to the whole claims team.

THE CLIENT'S ASK

- Improving the agent is slow manual work: read the misses, edit the instructions, rerun the evaluations, decide whether it helped.
- Put that loop in the product: an **improver agent** reads the results, proposes a change, waits for a person's approval and runs the evaluations. Code keeps or rolls back each change on held-out claims, and a judge model grades each explanation.
- Handlers sign in. Only a handler who can approve over the limit may run rounds, approve proposals or change settings.

HOW IT WORKS

- **Sign-in with no shared secret.** The backend checks each access token against the identity provider's published public keys (JWKS) and the expected audience, then finds the handler by the token's email. No token, an expired one or an unknown email → 401. A handler without the right → 403, and the dashboard says what they can't do and why.
- **Settings live in the database**: the approval limit, the accuracy and grade floors, the noise and tolerance per measure, the round's spending limit and the agents at once. Every change records who and when.
- **The configuration is versioned**: instructions, model and reasoning effort. Every run records the version it used.
- **The judge runs in the backend**, because the backend holds the handlers' decisions, so held-out reasons never leave it. It returns a score and a letter grade, plus two checks: same grounds, same section.
- **The improver** is a LangChain agent with four tools: read results, propose a change, run the evaluations, propose a new recipe. `HumanInTheLoopMiddleware` pauses on both proposals, with its own `PostgresSaver`, so a round waiting for approval survives a restart.
- **What the improver sees:** the evaluation set's misses and weakest explanations, with the handlers' reasons. From the held-out set it sees scores only, and a test checks for every held-out claim id and reason.
- **What it can't change:** the objectives, noise, tolerance, spending limit, allowed models, approval limit, evaluation sets or the judge.

```
first evaluations: 1 on the evaluation set, 2 on the held-out set
→ improver reads results → proposes a version → PAUSE
→ handler approves (or rejects with a reason → improver retries)
→ 3 evaluations of the new version → verdict by rule → recipe
```

The keep-or-roll-back rule, in code. Keep the new version only when it meets the objectives, at least one measure improves by more than its noise, and none gets worse by more than its tolerance, comparing the average of its two held-out runs with the current version. No grade, stopped evaluations or an unaffordable run → roll back. A round's spending limit ($0.50 at first) covers the improver, every evaluation and the judge.

HOW IT WAS PROVED

LIVE ROUND	RESULT
Round 1	first proposal rejected as too broad; the second, tied to the wording's conditions, approved. **Kept:** held-out accuracy 78% → 94%, grade 1.28 → 1.77, $0.199
Round 2	**Rolled back:** no measure improved by more than its noise · $0.102
Whole live pass	$0.333: claim agents, dispatch, improver and judge

Why held-out decides: the version kept in round 1 scored lower on the evaluation set (88% → 80%) while it rose on the held-out set. Tuning to the set it learns from would have kept the wrong change.

TRADE-OFFS AND WHEN TO SKIP

The judge's cost lands at the end of a run, so a round can end a few hundredths of a cent over its limit. **Skip it** when nobody signs in and the configuration is changed by hand.

LAYER 6

Security before the cloud

Builds on layers 1–5. Asked for by IT, with information security, before the app gets a public address.

THE CLIENT'S ASK: THREE FINDINGS

- **The agent sends nothing to identify itself.** Anyone who can reach the backend can record decisions or pay claims, and could record reviews in a handler's name.
- **Prompt injection.** A test claim hides an instruction in its story. The large model declines it; a smaller model approves it in full. A keyword list misses a reworded version. A second model reading each claim caught both versions and flagged none of the 25 real claims.
- **The app only runs on a laptop**, on a dev server, with a backend that seeds its own database at start.

DECISIONS PUT TO THE CLIENT

- **Which database the deployed app uses.** A serverless function can't reach the laptop, which breaks both the data and the sign-in key lookup. The FDE recommended building locally now and choosing a reachable database before the deploy.
- **What stands in for the hosted pooler locally.** A PgBouncer container in transaction mode, because like the hosted pooler it doesn't support prepared statements, so it really tests the change.
- **Every route belongs to exactly one group**, and a test enforces it: open (health), key or handler (shared reads), handler only (the dashboard's 18 routes), key only (the agent's routes).

HOW IT WORKS

- **An API key for the agent**, at least 32 random characters, sent in a header on every agent request and compared in constant time. An unset key refuses everything, so empty never matches empty. **The key never stands in for a handler:** handler actions refuse a key-only request with 401.
- **A hidden-instruction check is the graph's first node.** Before the claim agent's first model call, a second model reads every field the customer wrote and answers, with structured output, the reason first and then whether the claim tries to instruct whoever handles it.
- **On a hit, the claim agent's model never sees the claim.** It is recorded as a referral to a person, payout 0, with the check's reason. The check sits outside the versioned

configuration, so the improver can't change it or turn it off.

- **The agent's own API document lists exactly 8 operations.** A gateway makes a tool of every operation in the document it is given, so a narrow document keeps the dashboard, evaluations and sign-off out of the agent's reach.
- **The payment refuses an amount over the limit before it reads the claim**, with the same answer on any claim, even one that doesn't exist, so a policy test sees one consistent answer.
- **The sign-off pays, in one transaction**: handler, time and payment together. It needs no agent service, because the cloud function can't reach the laptop.
- **Ready for AWS Lambda:** the backend serves the built dashboard; a handler wraps the app with lifespan off, so nothing seeds on Lambda; on Lambda it opens one database connection per request (`NullPool`) with prepared statements off.

```
agent —API key→ agent's routes     handler —token→ handler's routes
claim → check node (second model reads the customer's text)
  hit   → refer to a person, payout 0, END (the agent never sees it)
  clean → claim agent → decision → pay (limit checked first)
```

HOW IT WAS PROVED

- Tests: no key, a wrong key and the right key on every agent route; a key-only request on every handler action; a flagged claim with the claim model never called.
- Live dispatch at N = 25: the check passed 25 claims and flagged only the planted one, with the reason that its story told the assistant to approve $4,800 and said a clause had been waived. 34 s, $0.031.
- Behind PgBouncer in transaction mode, the app served every claim with no connection kept between requests. The Lambda package is 20.3 MB zipped, under the 50 MB limit.

TRADE-OFFS AND WHEN TO SKIP

Only PgBouncer was tested; the hosted pooler is tried at deploy. In the same live evaluation the judge recorded no grades, an error it swallows that was still to be looked into. **Never skip this layer** for a product on the internet.

LAYER 7

Context, memory and a gateway

Builds on layers 1–6. Asked for by IT, with claims operations and data governance.

THE CLIENT'S ASK

- The agent's tools call the app directly, so the gateway's policy, which refuses any payment over $1,000, never sees what the agent does.
- What the agent needs to know is spread across systems, and outside facts such as named storms, strikes and product prices are nowhere it can reach. Data governance gathered them into 7 files, each naming its source system, owner and last review.
- Decision records fall short of the claims manual: no calculation, no maker, no manual version. And the agent forgets a customer between claims.

DECISIONS PUT TO THE CLIENT

- **How the agent finds sections once the documents move.** Putting both documents in the instructions would add about 300 lines to every model call and raise the cost of each claim several times over. Chosen: the context server also lists sections, the only option that keeps the agent's way of working.
- **Where the deployed data lives.** A managed Postgres and a managed user pool in the client's own AWS account, so the claims data stays inside it, with a database that is publicly reachable but requires TLS, because reaching a private one from Lambda needs a NAT gateway at about $32 a month.
- **Field names that differ between governance's dictionary and the app.** The FDE kept the app's names, did not edit governance's files, and raised it with data governance.

HOW IT WORKS

- **A context server over MCP** serves the 7 governed files through read-only tools: look up a term, describe the fields, get a plan, get the claim rules, search reference data, search and read documents. **Every answer names its file, with that file's system, owner and review date.**
- **Two routes to the same tools.** On the gateway route the agent calls AgentCore Gateway tools over streamable HTTP, signed with AWS SigV4; on the direct route it calls the app with its key and the context server unsigned. The model sees the same tool names and inputs on both.

- **Two kinds of refusal kept apart.** An app refusal comes back flagged as an error result; a gateway refusal raises an MCP error. The claim records which one refused.
- **A Cedar policy on the gateway**, a `permit` for each target's tools beside a `forbid` on payments over $1,000, in `ENFORCE` mode, refuses the call before it reaches the app. A refused payment holds the claim for a handler's sign-off.
- **Thin tools.** Reading the claim, the policy, the history and calculating the payout take no id from the model, so the model can act only on its own claim.
- **A complete decision record:** an approval needs a calculation on file for this attempt, and the payout must equal it. Each record holds its calculation, who decided (the agent and its version, or the security check), when, and the manual version.
- **Memory of each customer** with AgentCore Memory as the graph's LangGraph store. A node saves two events after each decision, in code, never at the model's choice, and never the story or the documents, because a story is where a customer's text could try to instruct the agent. Search runs only under that customer.

```
agent —MCP, SigV4→ AgentCore Gateway —Cedar policy→
   app target:     claim, policy, history, payout, decision, pay
   context target: terms, fields, plans, rules, reference, sections
answer + its file's system, owner and review date → step on the claim
```

HOW IT WAS PROVED

- A stand-in gateway in the tests names tools as the real one does, refuses a payment over $1,000, and keeps the request headers so a test can check the SigV4 signature.
- With the approval limit raised to $2,000, a $1,500 approval was refused by the stand-in, held, and paid once by a senior handler's sign-off.
- A restart past the memory node saves nothing twice. The full suite ended at 362 Python, 57 dashboard and 14 end-to-end tests.

TRADE-OFFS AND WHEN TO SKIP

The gateway, policy and memory on real AWS are written as notebook cells with a teardown, and were still to be run. While it exists the managed database costs about $0.60 a day. **Skip it** when the agent needs nothing beyond its own data and nothing must be remembered.

LAYER 8

The agent on a managed runtime

Builds on layers 1–7. Asked for by IT, with claims operations and the platform team, who own the runtime's files: "change the product to fit them".

THE CLIENT'S ASK

- A handler who thinks an approved payout is wrong can't tell the agent; they can only leave the claim waiting. They want to **send it back with a note**.
- The runtime wants port 8080, an invocations route and a ping route, and it ends a session after its idle timeout even if work is still running on a thread it doesn't know about.
- The service keeps one run in memory. With a session per dispatch, one session would stop another's work.

HOW IT WORKS

- **The answer lives in the app, not the request.** "Signed off" or "sent back" is read from the claim's record, so a resume, a retried call and the next dispatch all see the same answer.
- **Every "only once" is one conditional update of one row, checked by the row count.** A check followed by an insert doesn't stop two starts at Postgres's default isolation; a conditional update holds there, in SQLite and through a pooler.
- **Send back:** in one transaction the old decision is kept, the current one cleared and the note recorded. On resume the note reaches the model as a message from the handler, the earlier decision is forgotten so nothing can pay it, and the model-call allowance resets.
- **How to treat the note is fixed in code**, not in the instructions, so an improver's new version can't drop it: check the note against the wording and the manual, and refer the claim if the wording doesn't support it.
- **One entrypoint with `BedrockAgentCoreApp`.** Each piece of work is registered as an async task before the agent answers and completed in a `finally`, so ping says busy while work runs. A test showed registered work kept going for 2 minutes with no requests; unregistered work was killed a minute after the last one.
- **Nothing at import.** Every runtime session starts from one snapshot, so a key or connection made before the first request would be shared by all of them. The agent gets ready on its first request, behind a lock.
- **One slot for the agent's work**, a one-row table in the app, because on the runtime each session is its own process, so only the app can keep one run at a time.

- **Two routes to the agent behind one function**, local HTTP or the runtime, with three outcomes: not taken, taken, or unknown. Only "not taken" is undone; "unknown" asks the handler to reload. One attempt only, because a retry could start the work twice.

```
handler: send back + note → one transaction (old decision kept)
  → app asks the agent to resume (local HTTP or AgentCore Runtime)
  → agent: register async task → answer {"accepted": true}
  → take the claim (first caller wins) → Command(resume=note)
  → new decision → pay, or pause again → task complete → ping healthy
```

HOW IT WAS PROVED

- Two sign-offs at once: one pays, the other gets 409. A sign-off and a send-back at the same moment: exactly one goes through. A send-back the agent never received is worked by the next dispatch.
- Entrypoint tests: importing opens nothing, credentials are fetched once, ping is busy during work and healthy after.
- Live: a full dispatch, a claim signed off and paid, a claim sent back and decided again, and a second start refused while a dispatch ran.

TRADE-OFFS AND WHEN TO SKIP

A session per claim, tracing on the runtime and stopping a session from the dashboard were left for later. **Skip it** when the agent runs on your own servers.

LAYER 9

A copilot for the users

Builds on layers 1–8. A reviewed design, planned to be built next.

THE CLIENT'S ASK

- Handlers spend their day on the dashboard, and each task takes a few pages and a few clicks. Some questions take longer: what needs me today, why did the agent decide this, what does the wording say, are we meeting what we agreed.
- A sidebar on every page where they ask in plain English. It looks things up, starts work, sends back and signs off, **always as the handler who asked, with that handler's permissions**, showing each step.
- It answers the pilot's success criteria: at least 70% straight through on the held-out set, accuracy at or above the objective, and every payout over the limit approved by a person.

HOW IT WORKS

- **A sign-in gate first.** Each turn starts by asking the app who is signed in, with the turn's token. If the app refuses, the turn fails with no model call, no memory and no tool. The user the turn works for comes from that answer, never from the model or the message.
- **The token travels in the graph's context** (`context_schema`, read by each tool through `ToolRuntime`), never in its config, because LangGraph saves the config with every checkpoint. A test searches the checkpoints, steps, turns, memory and logs for it.
- **The copilot calls the app as the user, never with the agent's key.** The app refuses a request that carries the key, even with a valid sign-in, so the key can never stand in for a person.
- **A confirm node with** `interrupt()` before any sign-off. It checks in code that this user may sign off this claim before it pauses. The confirmation card is filled by the app from its records, never from what the model wrote. The pause survives a reload or another day.
- **The app owns the turn's state.** The checkpoints hold the conversation; the app's tables hold what the sidebar shows, and the sidebar reads only the app.
- **Waiting and figures stay in code.** A tool waits for a run by reading it every 5 seconds for up to 15 minutes, with no model calls. The success criteria are worked out in code and handed to the model as a finished table.
- **A second AgentCore app** beside the claim agent, one runtime session per conversation, and its own memory of each user's preferences.

```
gate → remember → model ⇄ tools
                    ├→ confirm (interrupt) → tools
                    └→ save → END
sidebar polls the app about once a second while a turn works
```

HOW IT WILL BE PROVED

- A refused sign-in ends the turn before any model call; a fresh copilot answering a saved pause pays once and repeats nothing; a handler without the right gets the reason and no card.
- Live: the five handover questions in one conversation, every figure checked against the scoreboard, and the conversation picked up after a restart.

TRADE-OFFS AND WHEN TO SKIP

No streaming, because the app on Lambda can't stream; progress shows as step cards instead. One agent, one graph: not built for agents handing work to each other. **Skip it** when users never need to ask or act through chat.

THE END

What carries to the next client

The client, the documents and the rules change. These patterns don't.

PATTERN	LAYER
The backend owns every rule; the agent can only ask	1
A refusal from the backend is feedback that steers the agent	2
A wait for a person is a node of its own, with checkpoints after every step	3
Every write is safe to repeat; every "only once" is a conditional update	3, 8
Evaluate on copies; the answer key is out of the agent's reach	4
A version is kept on measured held-out scores, never on the model's opinion	5
The agent has its own key and its own narrow API; the key never stands in for a person	6, 9
Untrusted text is screened before the model sees it, and never saved to memory	6, 7
Tools behind a gateway with a policy; every answer names its source and owner	7
The answer lives in the app; nothing runs at import	8
A user's agent acts with that user's sign-in, and figures come from code	9

What was proved, and what wasn't. Layers 1 to 8 were built with their tests and run live, apart from the AWS half of layer 7 (the real gateway, policy and memory), which was written as notebook cells still to be run. Layer 9 is a reviewed design. The numbers in this article come from those live runs on a 25-claim example queue.

Want an agent platform like this around your own tools? VerBodh builds secure MCP servers and AI agents.

verbodh.com